



**INSTITUTO
FEDERAL**

Goiano

Campus
Trindade

Sistemas Distribuídos

gRPC com Node.js

Protocol Buffers · HTTP/2 · Contrato-primeiro

Prof. Dr. Renato de Sousa Gomide

Objetivos da aula

- Entender **gRPC** como evolução multiplataforma do **RPC**
- Escrever um **contrato** em **Protocol Buffers** (`.proto`)
- Implementar **servidor** e **cliente** em **Node.js** com `@grpc/grpc-js`
- Conhecer os **quatro modos** de comunicação (unário e streaming)
- Comparar **gRPC × REST** e saber **quando** usar cada um

Retomando: RPC como conector

Um **componente** manda uma requisição para **outro componente** usando a sintaxe de uma **chamada local**.

Java RMI resolveu isso — mas **só para Java**, com contrato preso a uma **interface Java**.

Sistemas reais são **heterogêneos**: um serviço em Go, outro em Python, o cliente em Node.js.

gRPC é RPC **independente de linguagem**, com contrato **explícito** e transporte **HTTP/2**.

O que é gRPC?

- Framework de **RPC** de alto desempenho, criado no **Google** e hoje mantido na **CNCF**
- O "g" originalmente vem de **Google**; hoje muda a cada release
- Três pilares:
 - **Protocol Buffers** — linguagem de contrato (**IDL**) e formato **binário**
 - **HTTP/2** — multiplexação, cabeçalhos comprimidos, streaming
 - **Geração de código** — stubs de cliente e servidor em dezenas de linguagens

Contrato-primeiro

Em REST o contrato costuma ser **documentação**. Em gRPC o contrato é **código-fonte**:

```
aluno.proto → gera → stub cliente + esqueleto servidor
```

- O **mesmo** `.proto` serve Node.js, Java, Go, Python, C#, Rust...
- Mudou o contrato? Os dois lados **recarregam o mesmo arquivo**
- Erro de nome ou tipo aparece **cedo**, não em produção

Analogia: o `.proto` é a **planta baixa**. Pedreiro e eletricista trabalham a partir dela, sem combinar nada por telefone.

Protocol Buffers (proto3)

src/contrato/aluno.proto

```
syntax = "proto3";

service AlunoService {
    rpc criarAluno (Aluno) returns (Aluno) {}
    rpc getAlunoByMatricula (AlunoMatricula) returns (Aluno) {}
}

message Aluno {
    int32 matricula = 1;
    string email = 2;
    string nome = 3;
}

message AlunoMatricula {
    int32 matricula = 1;
}
```

Anatomia do `.proto`

Elemento	Significado
<code>syntax = "proto3"</code>	Versão da linguagem — sempre declare
<code>service</code>	Conjunto de operações remotas
<code>rpc nome (Entrada) returns (Saída)</code>	Uma operação: um parâmetro de entrada, um de saída
<code>message</code>	Estrutura de dados trafegada
<code>= 1</code> , <code>= 2</code>	Números de campo (<i>field numbers</i>)

Toda `rpc` recebe **uma** mensagem e devolve **uma** mensagem. Precisa de mais dados? Coloque-os **dentro** de uma `message`.

Os números de campo são o contrato real

No formato binário **o nome do campo não trafega** — só o **número**.

```
string email = 2;    // no fio: campo 2, tipo string
```

- **Nunca reutilize** um número já usado: dados antigos seriam lidos com o significado errado
- **Renomear** um campo é compatível; **mudar seu número quebra** compatibilidade
- Números **1–15** ocupam **1 byte** — reserve-os para campos frequentes
- Campo removido → marque com `reserved` para impedir reuso

Tipos e valores padrão em proto3

Categoria	Tipos
Numéricos	<code>int32</code> , <code>int64</code> , <code>uint32</code> , <code>float</code> , <code>double</code>
Texto/bytes	<code>string</code> , <code>bytes</code>
Lógico	<code>bool</code>
Compostos	<code>message</code> , <code>enum</code> , <code>repeated</code> (lista), <code>map<k,v></code>

Em proto3 **todo campo é opcional** e tem **valor padrão**: `0` para números, `""` para string, `false` para bool.

Não há como distinguir "campo ausente" de "campo igual a zero" — a menos que use `optional` explicitamente.

Os quatro modos de comunicação

Modo	Assinatura	Uso típico
Unário	<code>rpc f (Req) returns (Res)</code>	Requisição/resposta clássica
Server streaming	<code>returns (stream Res)</code>	Notificações, resultados paginados
Client streaming	<code>(stream Req) returns (Res)</code>	Upload, envio de lote
Bidirecional	<code>(stream Req) returns (stream Res)</code>	Chat, telemetria contínua

Nosso projeto usa apenas o modo **unário** — mas a mesma stack suporta os outros trocando a palavra `stream`.

Stack do projeto

Pacote	Papel
<code>@grpc/grpc-js</code>	Implementação de gRPC em JavaScript puro (sem addon nativo)
<code>@grpc/proto-loader</code>	Lê o <code>.proto</code> em tempo de execução e monta os objetos de serviço

```
npm install
```

Alternativa: gerar código **estaticamente** com `protoc` / `grpc-tools`. Mais rápido e tipado; aqui usamos o carregamento **dinâmico**, mais simples para aprender.

Carregando o contrato

Mesmo bloco no **servidor** e no **cliente**:

```
const grpc = require("@grpc/grpc-js");
const protoLoader = require("@grpc/proto-loader");

const PROTO_PATH = __dirname + "../contrato/aluno.proto";

const packageDefinition = protoLoader.loadSync(PROTO_PATH, {
  keepCase: true,
  defaults: true,
  oneofs: true,
});

const protos = grpc.loadPackageDefinition(packageDefinition);
```

`protos.AlunoService` é o serviço declarado no `.proto`.

Opções do proto-loader

Opção	Efeito
<code>keepCase: true</code>	Mantém os nomes como escritos no <code>.proto</code> (sem converter para <i>camelCase</i>)
<code>defaults: true</code>	Preenche campos ausentes com o valor padrão do tipo
<code>oneofs: true</code>	Expõe qual campo de um <code>oneof</code> foi preenchido
<code>longs</code> , <code>enums</code>	Controlam como <code>int64</code> e <code>enum</code> chegam ao JavaScript

Cliente e servidor **devem usar as mesmas opções**, senão os nomes dos campos divergem entre os lados.

O servidor — handlers

src/server/server.js

```
function criarAluno(call, callback) {  
  const { email, nome, matricula } = call.request;  
  console.log('criarAluno\nREQUEST:');  
  console.log(matricula, nome, email);  
  
  callback(null, {  
    matricula: 1,  
    email: "aluno@aluno.com",  
    nome: "Jovem"  
  });  
}
```

- **call.request** — a mensagem recebida, já desserializada
- **callback(erro, resposta)** — padrão Node.js: **erro primeiro**, `null` quando deu certo

O servidor — publicação

```
const server = new grpc.Server();

server.addService(protos.AlunoService.service, {
  criarAluno,
  getAlunoByMatricula,
});

await server.bindAsync("0.0.0.0:4000",
  grpc.ServerCredentials.createInsecure(),
  (err, port) => {
    if (err) console.error(err);
    else console.log("Server is running");
  });
```

As **chaves** do objeto passado a `addService` devem bater **exatamente** com os nomes das `rpc` no `.proto`.

`0.0.0.0` e credenciais

- `0.0.0.0:4000` — escuta em **todas** as interfaces de rede, não só `localhost`
- `createInsecure()` — sem **TLS**: tráfego em **texto claro**
 - aceitável em **laboratório** e dentro de uma rede confiável
 - em produção use `grpc.ServerCredentials.createSsl(...)`

Diferente do RMI, aqui a porta é **fixa e única** (4000) — muito mais amigável a firewall e contêineres.

O cliente

src/client/client.js

```
const client = new protos.AlunoService(
  "localhost:4000",
  grpc.credentials.createInsecure()
);

const aluno = { email: "criar@aluno.com", nome: "NovoAluno", matricula: "123456" };

client.criarAluno(aluno, function (err, response) {
  console.log("Response:", response);
});

client.getAlunoByMatricula({ matricula: 123 }, function (err, response) {
  console.log("Response:", response);
});
```

O stub `client` **parece** um objeto local — mesma ideia do stub do RMI.

Dois detalhes do cliente

1. `matricula: "123456"` é uma **string** para um campo `int32`. O protobuf **converte** o que consegue; um valor não numérico viraria erro de serialização. **Respeite os tipos do contrato.**
2. As duas chamadas são **assíncronas** e usam **callback**. A ordem dos `console.log` **não** é garantida — as respostas chegam quando chegam.

Para usar `async/await`, envolva a chamada em uma `Promise` ou use uma biblioteca de *promisify*.

Executando

Instale as dependências na raiz do projeto:

```
npm install
```

Servidor:

```
node src/server/server.js # ou: npm run server
```

Cliente, em **outro terminal**:

```
node src/client/client.js # ou: npm run client
```

Suba o **servidor primeiro**. Sem ele, o cliente falha com `UNAVAILABLE`.

Códigos de status do gRPC

gRPC **não** usa códigos HTTP na aplicação — tem os seus:

Código	Quando ocorre
OK (0)	Sucesso
INVALID_ARGUMENT (3)	Dados da requisição inválidos
NOT_FOUND (5)	Recurso inexistente
UNAUTHENTICATED (16)	Credencial ausente ou inválida
UNAVAILABLE (14)	Servidor fora do ar (erro mais comum no laboratório)

```
callback({ code: grpc.status.NOT_FOUND, message: "Aluno não encontrado" });
```

gRPC × REST

	gRPC	REST
Contrato	<code>.proto</code> (obrigatório)	OpenAPI (opcional)
Formato	Protobuf binário	JSON texto
Transporte	HTTP/2	HTTP/1.1
Legibilidade	Precisa de ferramenta	Legível por humanos
Streaming	Nativo, bidirecional	Limitado (SSE, WebSocket)
Navegador	Só via grpc-web + proxy	Nativo
Ponto forte	Comunicação entre serviços	API pública

Quando usar cada um

gRPC: comunicação **interna** entre microsserviços, baixa latência, alto volume, times em linguagens diferentes, streaming.

REST/JSON: API **pública**, consumo direto por **navegador**, integração com terceiros, facilidade de depuração com `curl`.

Não é escolha excludente: é comum expor **REST na borda** (*gateway*) e usar **gRPC internamente**.

Limitações do gRPC

- **Navegador** não fala gRPC direto — exige **grpc-web** e um proxy
- Payload **binário**: não dá para inspecionar com `curl`; use `grpcurl` ou **Postman**
- Curva de aprendizado do **protobuf** e do *tooling*
- Continua **síncrono e acoplado no tempo** como RPC/RMI: precisa do servidor **no ar**
 - para desacoplar, use **mensageria** (Kafka, RabbitMQ) — próximo tema do curso

Resumo

- **gRPC** = RPC **multiplataforma** sobre **HTTP/2** com contrato em **Protocol Buffers**
- O `.proto` define `service`, `rpc` e `message` — os **números de campo** são o contrato no fio
- `@grpc/proto-loader` carrega o contrato **em tempo de execução**;
`@grpc/grpc-js` faz o transporte
- Servidor: `addService` + `bindAsync`; cliente: **stub** com callback `(err, response)`
- Quatro modos: **unário** e três formas de **streaming**
- Comparado ao **RMI**: mesma ideia de stub, mas **sem amarração a uma linguagem**

Referências

- [Introdução ao gRPC desenvolvendo uma aplicação com Node.js](#) — base deste projeto
- [gRPC](#) — site oficial e [gRPC for Node.js](#)
- [Language Guide \(proto 3\)](#)
- [Qual é a diferença entre gRPC e REST?](#)